PM – Lucrarea 5

Această lucrare tratează o instrucțiune specifică arhitecturii Intel x86, denumită CPUID, care returnează două seturi de informații despre procesorul pe care este executată: un set de informații de bază (basic processor information) și un set de informații extinse (extended processor information). Această instrucțiune este descrisă în detaliu în documentația Intel: https://www.microbe.cz/docs/CPUID.pdf#page=15&zoom=auto,-150,721

4.1 Modul de funcționare al instrucțiunii CPUID

Incepând cu anul 1993, odată cu lansarea procesoarelor Intel Pentium și SL-enhanced 486, a fost implementată instrucțiunea CPUID care oferă detalii despre procesor. Valorile returnate de CPUID depind de valoarea de intrare specificată ca parametru în registrul EAX, înainte de a apela CPUID.

Instrucțiunea CPUID suportă două seturi de funcții:

- Un set pentru informații de bază despre procesor (EAX=0,1,…,CPUID(0).EAX) (fig. 4.1)
- Un set pentru informații extinse despre procesor (EAX=80000000h,…,CPUID(80000000).EAX)

Mai multe detalii despre informațiile returnate de CPUID pot fi găsite în documentația Intel: https://www.microbe.cz/docs/CPUID.pdf#page=13&zoom=auto,-150,707

Familia de procesoare Intel pune la dispoziție o metodă directă prin care se poate afla dacă arhitectura internă a procesorului pe care dorim să rulăm un program cu instrucțiunea CPUID permite să execute acest cod. Această metodă presupune utilizarea flag-ului de pe poziția bitului 21 din registrul EFLAGS. Dacă programul scris de utilizator reușește să modifice acest flag, instrucțiunea CPUID poate fi executată.

Pentru a accesa flag-urile din registrul EFLAGS, trebuie utilizate instrucțiunile assembly PUSHF și POPF pentru operații pe 16 biți sau PUSHFD și POPFD pentru operații pe 32 de biți, după cum urmează:

POPF – copiază un cuvânt (word – 16 biti) de la vârful stivei și îl stochează în cei 16 biți mai puțin semnificativi din registrul EEFLAGS;

PUSHF – salvează cei 16 biți mai puțin semnificativi din EFLAGS în stivă;

POPFD – copiază un double word (32 de biți) de la vârful stivei și îl stochează în registrul EFLAGS;

PUSHFD – salvează conținutul EFLAGS (32 de biți) în stivă.

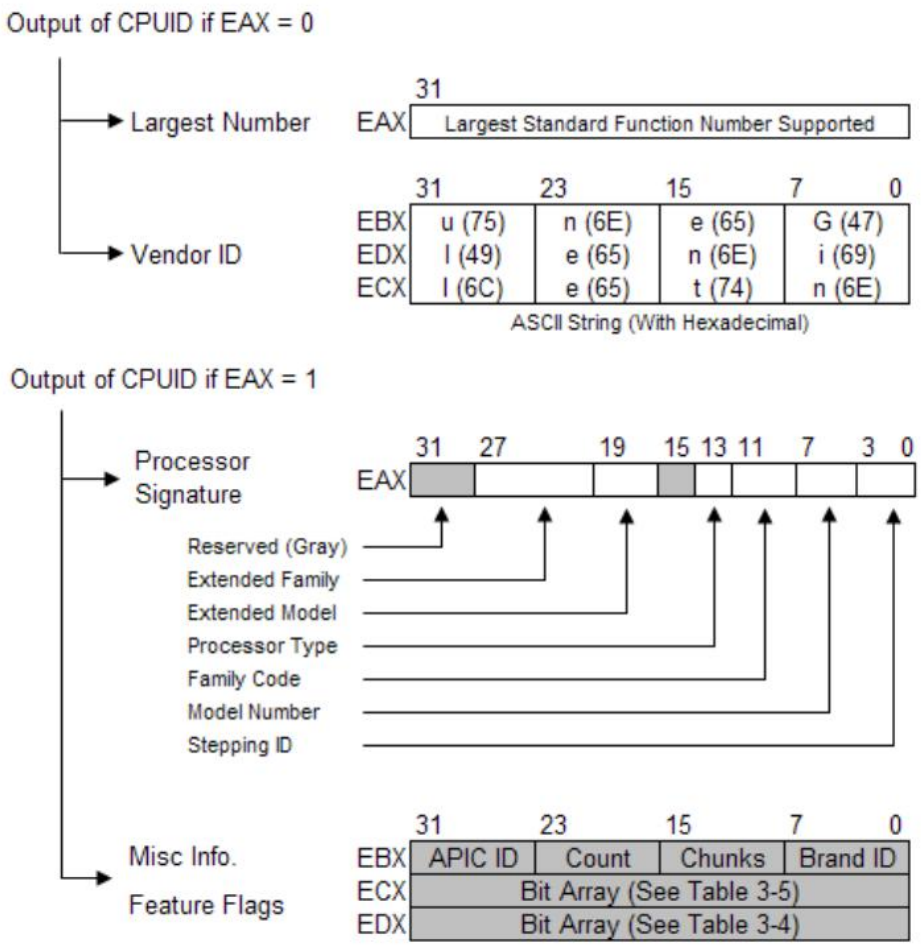

Fig. 4.1 Informațiile returnate de CPUID pentru EAX=0 și EAX=1

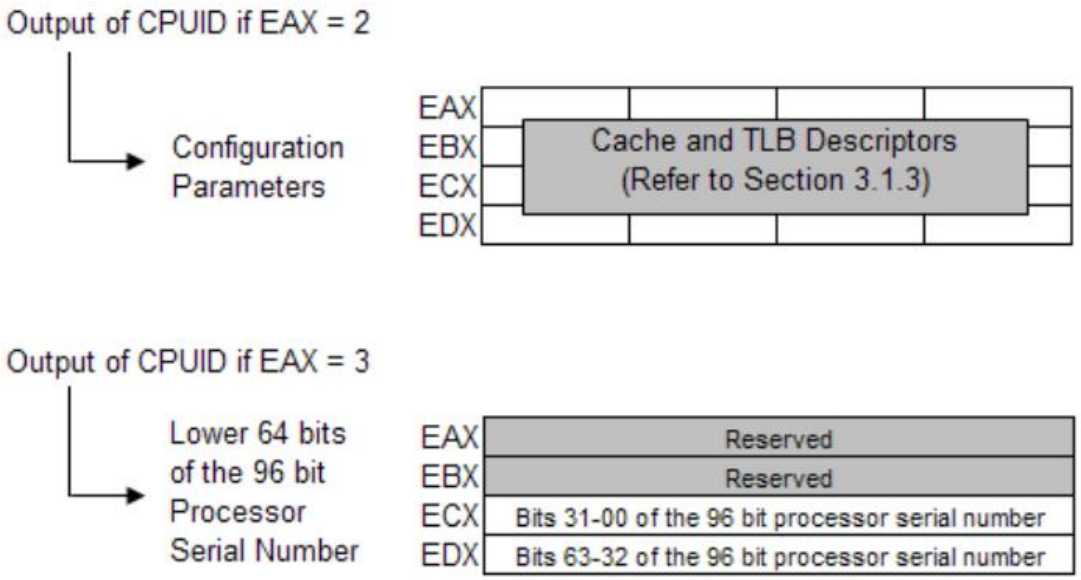

Fig. 4.2 Informațiile returnate de CPUID pentru EAX=2 și EAX=3

4.2 Exemplu de integrare cod assembly într-un program C / C++

Instrucțiunile în limbajul de asamblare x86 pot fi integrate într-un program C / C++ folosind directiva *__asm* după cum urmează:

Exemplul 1: Scrierea valorii unei variabile pe 4 bytes (32 de biti) într-un registru:

```
unsigned int a = 7;
__asm {
        mov eax, a     ;registrul EAX ia valoarea 7
}
```

Exemplu 2: Citirea valorii dintr-un registru și copierea acesteia într-o variabilă pe 4 bytes (32 de biți):

```
unsigned int a  = 0;
unsigned int b;
__asm {
        mov eax, a    ;copiem valoarea 0 în registrul EAX, pentru a apela CPUID în cazul EAX = 0
        cpuid
        mov b, ebx ;valoarea registrului EBX, suprascris de apelul instrucțiunii CPUID, este copiată în variabila b
}
```

4.3 Exemplu de program care returnează informații despre procesor, folosind CPUID și cod assembly integrat în C++

În această secțiune este prezentat un program C++ cu cod assembly integrat, pentru obținerea unor informații despre procesor. Pentru a rula acest program, se poate folosi un IDE de C++. Recomandăm utilizarea Visual Studio Community 2019 și crearea unui nou proiect de tipul “Empty Project”, ca în figura 4.3. Codul sursă trebuie salvat într-un fișier cu extensia .cpp.

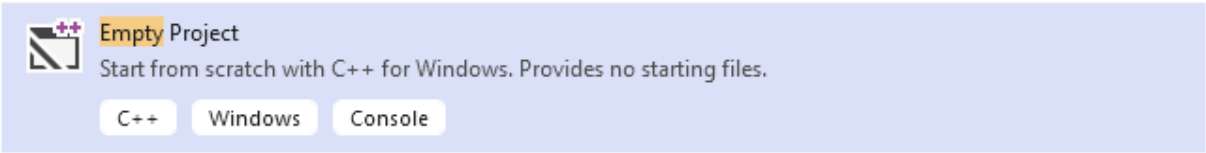

Fig. 4.3 Visual Studio Community 2019 - Empty Project

```
#include <windows.h>
#include <iostream>
using namespace std;
#include <tchar.h>
#include <psapi.h>
#include <cstdlib>
#include <stdint.h>
#include <intrin.h>
#include <synchapi.h>
#include <winbase.h>
#include <processthreadsapi.h >


void procInfo() {
	unsigned long int eflags1, eflags2;
	char vendorID[13];
	unsigned long int success = 0;
	unsigned long int modelNum, FamilyCODE, ExtMODE, procTYPE, extFam, start;
	unsigned long int brandID, prop, proccount, ca, cb, cc, cd;

	__asm {
		pushfd
		pop eax; preluare EFLAGS

		mov eflags1, eax  ; reținem valoarea inițială a lui eflags

		xor eax, 00200000h; modificam bitul 21
		mov eflags2, eax  ; reținem valoarea modificată

		push eax
		popfd; scriem valoarea modificată in EFLAGS

		pushfd
		pop eax; preluare EFLAGS

		cmp eax, eflags2
		je SUCCESS
		jmp CONTINUE
		SUCCESS : mov success, 1
		mov eax, 00h
		cpuid   ; setare info cerută în eax


		CONTINUE : mov eax, eflags1; readucem EFLAGS la valoarea inițială

		mov eax, 0 ; CPUID va returna Vendor ID în regiștrii EBX, EDX si ECX
pentru parametrul de intrare EAX = 0
		cpuid
		mov DWORD PTR vendorID, ebx
		mov DWORD PTR vendorID + 4, edx
		mov DWORD PTR vendorID + 8, ecx

		mov eax, 1 ; CPUID va returna informațiile denumite “Processor Signature”
(vezi fig. 4.1) pentru parametrul de intrare EAX = 1
		cpuid
		mov modelNum, eax
		and modelNum, 0F0h
		mov FamilyCODE, eax
		and FamilyCODE, 0F00h
		mov procTYPE, eax
		and procTYPE, 3000h
		mov ExtMODE, eax
		and ExtMODE, 0F0000h
```

```
            mov extFam, eax
            and extFam, 0FF00000h

            mov brandID, ebx
            and brandID, 0FFh

            mov prop, edx
            and prop, 0Fh

            mov eax, 1
            mov ecx, 0
            cpuid
            mov proccount, ebx
            shr proccount, 16
            and proccount, 0FFh

            mov eax, 2  ; CPUID va returna parametrii de configurare ai memoriei
Cache pentru parametrul de intrare EAX = 2
            cpuid
            mov ca, eax  ; în urma apelării instrucțiunii cpuid, rezultatul returnat
în registrul eax este salvat în variabila ca
            mov cb, ebx  ; rezultatul returnat în registrul ebx este salvat în
variabila cb
            mov cc, ecx
            mov cd, edx
      }

      if (success != 0) {
            printf("EFLAGS initial si cel modificat sunt egale.\n");

      }
      else {
            printf("EFLAGS initial si cel modificat nu sunt egale.\n\n");
      }

      vendorID[12] = '\0';
      cout << "Vendor ID: " << vendorID << "\n\n";

      modelNum >>= 4;
      FamilyCODE >>= 8;
      procTYPE >>= 12;
      ExtMODE >>= 16;
      extFam >>= 20;

      cout << "Model Number: " << modelNum << "\n";

      cout << "Family Code: " << FamilyCODE << "\n";

      cout << "Extended Mode: " << ExtMODE << "\n";

      cout << "Processor Type: " << procTYPE << "\n";

      cout << "Extended Family: " << extFam << "\n";

      brandID >>= 7;

      cout << "Brand ID: " << brandID << "\n\n";

      cout << "Flags FPU, VME, DE, PSE: " << prop << "\n\n";

      cout << "No. procesoare logice: " << proccount << "\n\n";
```

```
	cout << "Info. despre cache: \n" << std::hex << ca << "\n" << std::hex << cb << "\n" << std::hex << cc << "\n" << std::hex << cd << "\n\n";


}


int _tmain(int argc, _TCHAR* argv[])
{
	procInfo();
}
```

In urma rulării acestui program pe un laptop Lenovo produs în 2019-2020, cu procesor din generația 10, Intel Core i5-1035G1 @ 1.00GHz, 4 cores, 8 logical processors, obținem mesajul "EFLAGS inițial și cel modificat sunt egale" (fig. 4.4), ceea ce indică faptul că bitul 21 din registrul EFLAGS nu a putut fi modificat, deci informațiile returnate de CPUID nu sunt relevante. Însă, în funcție de sistemul desktop / laptop pe care îl deține fiecare dintre voi, rezultatul rulării programului poate fi diferit.

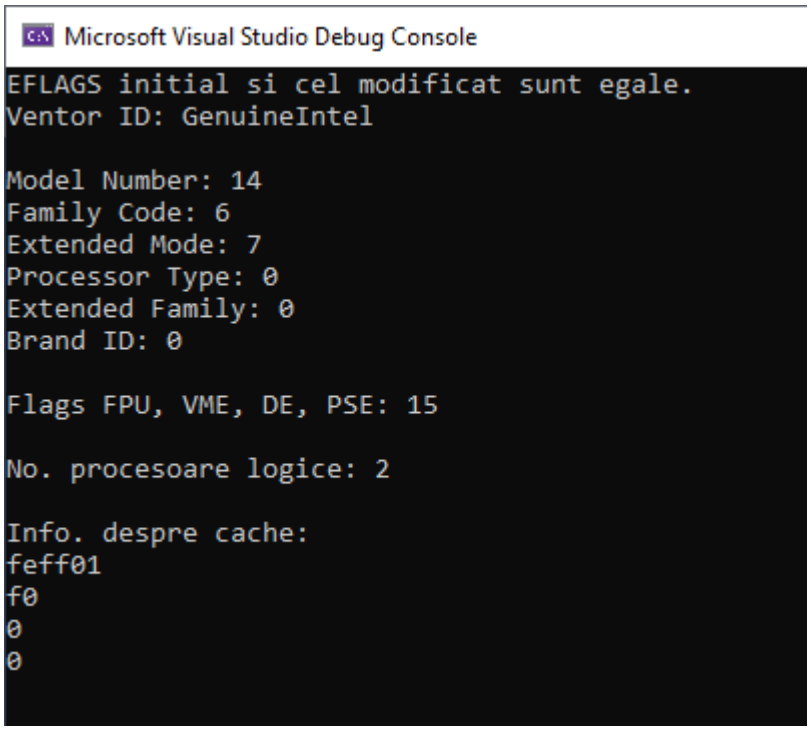

Fig. 4.4 Rezultat rulare program pe un procesor Intel i5 din generația 10

4.4 Cerințe și fișă de laborator

Pasul I: Compilați și executați programul dat ca exemplu în secțiunea 4.3 a lucrării de laborator în Visual Studio Community sau un alt IDE pentru C / C++ la alegerea voastră. Se cer două capturi de ecran: una care să conțină rezultatele afișate de program în consolă și cealaltă care să arate tipul de procesor de pe sistemul vostru și sistemul de operare pe care ați rulat codul. Pentru a doua captură de ecran intrați în Windows în Control Panel -> System ți căutați informațiile procesorului, similar cu figura 4.5.

Pasul II: Dacă programul a returnat mesajul "EFLAGS initial si cel modificat nu sunt egale", verificați dacă valoarea numărului de procesoare logice returnată de program coincide cu numărul de procesoare logice afișate în System. Includeți un printscreen care să arate acest aspect.

Fig. 4.5 Informatii sistem

Pasul III: Răspundeți la următoarele întrebări în legătură cu programul executat:

1. Folosind documentația Intel, explicați de ce se aplică următoarele operații pe biți, constând în deplasarea spre dreapta cu 4, 8, 12, 16, respectiv 20 de poziții, a variabilelor din porțiunea de cod de mai jos:

```
vendorID[12] = '\0';
cout << "Vendor ID: " << vendorID << "\n\n";

modelNum >>= 4;
FamilyCODE >>= 8;
procTYPE >>= 12;
ExtMODE >>= 16;
extFam >>= 20;

cout << "Model Number: " << modelNum << "\n";

cout << "Family Code: " << FamilyCODE << "\n";

cout << "Extended Mode: " << ExtMODE << "\n";

cout << "Processor Type: " << procTYPE << "\n";

cout << "Extended Family: " << extFam << "\n";
```

2. Explicați care este rolul instrucțiunilor: `pushfd` si `pop eax`.

3. Folosind documentația Intel furnizată, scrieți care este registrul procesorului care va conține informațiile Extended Family și Extended Model în urma apelării instrucțiunii CPUID și care sunt pozițiile binare revendicate de fiecare dintre acestea.

4. Folosind documentația Intel furnizată, scrieți care este registrul procesorului care va contine informatiile APIC ID si Count în urma apelării instrucțiunii CPUID și care sunt pozițiile binare revendicate de fiecare dintre acestea.

5. Folosind documentația Intel furnizată, scrieți care ar trebui să fie continutul binar al registrului EAX in urma apelului instrucțiunii CPUID pentru procesoarele Intel 486 SX.

6. Folosind documentația Intel furnizată, scrieți care ar trebui să fie continutul binar al registrului EAX in urma apelului instrucțiunii CPUID pentru procesoarele Intel Pentium Pro, precum și pentru procesoarele Intel Core i7.

7. Explicați la ce este folosită variabila `unsigned long int brandID` din codul exemplu. Care biți, din care registru, vor fi salvați în această variabilă?